Algorithmic Efficiency & Recursion Toolkit (AERT)

School of Engineering & Technology (SOET)

Program: B. Tech CSE (AI & ML), 2nd SEM

Course Title: Data Structure

Course Code: ETCCDS202

Unit Title: Foundations & Algorithmic Analysis

Submitted by: Samarth Sharma

Roll no: 2501730279

Section: A

Submission Date:

INTRODUCTION

The assignment involves creating a small Python toolkit called **Algorithmic Efficiency & Recursion Toolkit (AERT)**. The main goal of this toolkit is to understand how recursion works, study the efficiency of algorithms, and relate theoretical ideas such as **Big-O notation** and **algorithm design paradigms** with practical programming.

The toolkit contains several implementations including:

- Implementation of a **Stack ADT**
- A **Recursive Factorial** Function
- Two versions of **Fibonacci** (*naive and memo*)
- **Tower of Hanoi** with the step by step trace for N=3
- **Recursive Binary Search**
- **Complexity** and **paradigm** analysis

This report explains how each problem works and analyzes its time and space complexity in simple terms.

Part A- Stack ADT

What is an ADT?

An **Abstract Data Type (ADT)** describes the operations that can be performed on a type of data, without explaining the internal way those operations are carried out.

For instance, a **Stack ADT** includes operations such as **push**, **pop**, **peek**, **is_empty**, and **size**. However, it does not define how the stack is actually built. The stack could be implemented using different data structures, such as an **array** or a **linked list**.

In simpler terms:

- ADT focuses on what operations the data supports.
- A data structure focuses on how those operations are implemented internally.

Why Stack Operations Are O(1)?

Stack operations like push and pop always happen at one end (the top). They do not require searching, shifting, or traversing elements. Therefore, the time taken does not depend on how many elements are inside the stack. So:

- push → O(1)
- pop → O(1)
- peek → O(1)
- is_empty → O(1)
- size → O(1)

All operations run in constant time.

How stack is used in my program?

In my program, I used the Stack ADT in the Tower of Hanoi implementation. Every time a disk move is made, that move is pushed into the stack. After completing the recursion, I used stack.size() to display the total number of moves stored. This shows practical usage of the Stack ADT along with recursion. It also reflects how recursion itself internally uses a call stack.

<u>Part B – Factorial & Fibonacci (Recursion + Complexity)</u>

1. Recursive Factorial

In my program, the **factorial function** is implemented using **recursion**. The function first checks whether the input value is negative. If it is negative, the function returns **"Invalid input"**. If the value of **n** is **0 or 1**, the function returns **1**, which acts as the **base case** for the recursion. For any value greater than 1, the function calls itself and calculates the factorial using the recursive formula:

n × factorial(n − 1)

This ensures the function reduces the problem size at every step and stops at the base case.

Time Complexity:

- Each call reduces n by 1.
- Total recursive calls are proportional to n.
- **Big-O: O(n)**
- **Big-Θ: Θ(n)**
- Best case: O(1) (when n = 0 or 1)

Space Complexity:

- Recursion depth = n
- **O(n)** (due to call stack)

2. Fibonacci – Naive Recursive Version

The **naive Fibonacci function** is based on the recursive formula:

$F(n) = F(n - 1) + F(n - 2)$

with the base cases **$F(0) = 0$** and **$F(1) = 1$**.

In my implementation, I used a **global variable called naive_calls** to keep track of how many recursive calls are made during the calculation.

This approach becomes inefficient because the function repeatedly calculates the same Fibonacci numbers. For instance, while finding **F(5)**, the value **F(3)** is computed multiple times. As the value of **n** grows larger, these repeated calculations increase quickly, which makes the algorithm significantly slower.

Time Complexity:

- Recurrence: $\mathbf{T(n) = T(n - 1) + T(n - 2)}$
- Growth is exponential.
- **Big-O: $O(2^n)$**
- **Big-Θ: $\Theta(2^n)$**

Space Complexity:

- Maximum recursion depth = n
- **O(n)**

Fibonacci – Memoized Version

In the **memoized version**, I used a **dictionary** to store Fibonacci values that have already been calculated. Before computing **F(n)**, the function first checks whether the value is already present in the dictionary. If the value exists, the function simply returns the stored result instead of calculating it again.

This method prevents repeated calculations and greatly improves the efficiency of the algorithm. To compare its performance with the naive approach, I used a variable called **memo_calls** to count the number of recursive calls made in this version.

Time Complexity:

- Each Fibonacci value is computed only once.

- **Big-O: O(n)**
- **Big-Θ: Θ(n)**

Space Complexity:

- Recursion stack → O(n)
- Memo storage → O(n)
- Overall → **O(n)**

Call Counter Observation

From the output, it is clearly visible that:

- The naive version makes a very large number of recursive calls.
- The memoized version makes significantly fewer calls.

This shows how memoization reduces time complexity from exponential $O(2^n)$ to linear O(n).

<u>Part C – Tower of Hanoi</u>

Why number of moves becomes 2n−12^n - 12n−1?

For the Tower of Hanoi problem, the recurrence relation can be written as:

$T(n) = 2T(n - 1) + 1$

This happens because:

- First, we move (n − 1) disks from the source peg to the auxiliary peg.
- Then we move the largest disk to the destination peg.
- Finally, we move the (n − 1) disks from the auxiliary peg to the destination peg**.**

When this recurrence relation is solved, the total number of moves becomes:

$T(n) = 2^n - 1$

This means that to move n disks, the puzzle requires $2^n - 1$ moves.

Time Complexity:
Since the number of moves increases exponentially as n grows, the time complexity is $O(2^n)$.

Space Complexity:
The recursion can go as deep as n levels in the call stack, so the space complexity is O(n).

Part D – Recursive Binary Search (Divide & Conquer + Recurrence)

In my program, recursive binary search is used to find an element in a sorted array. The function takes four parameters: the array, the value to be searched (key), and the low and high indices of the current search range.

At every step of the algorithm:

- The middle index of the current range is calculated.
- If the middle element is equal to the key, the function returns its index.
- If the key is smaller than the middle element, the function continues searching in the left half of the array.
- If the key is greater, the search continues in the right half.
- If low > high, the function returns −1, which indicates that the element is not present in the array.

The program also handles special cases properly, such as when the array is empty.

Time Complexity (Using Recurrence)

In binary search, the search space becomes half at every recursive call. Therefore, the recurrence relation is:

$T(n) = T(n/2) + O(1)$

Here:

- $T(n/2)$ represents the recursive search in half of the array.
- $O(1)$ represents the constant amount of work done at each step (calculating the middle index and comparing values).

When this recurrence is solved, the overall time complexity becomes $O(\log n)$. This happens because the array keeps getting divided into halves until only one element remains.

Best, Average, and Worst Case

- **Best Case:**
  The element is found at the first middle position.
  Time Complexity = $O(1)$

- **Average Case:**
  The element is found after a few divisions of the array.
  Time Complexity = O(log n)
- **Worst Case:**
  The element is either not present or found after the maximum number of divisions.
  Time Complexity = O(log n)

So overall:

- **Big-O: O(log n)**
- **Big-Θ: Θ(log n)**
- **Big-Ω: Ω(1)**

Space Complexity

Since recursion is used, each recursive call is stored in the call stack. The maximum recursion depth is log n, so the space complexity is O(log n).

Part E – Algorithm Paradigm Reflection

Binary Search – Divide & Conquer: Binary search follows the Divide & Conquer strategy because it divides the problem into smaller parts at every step. Instead of checking every element, it finds the middle element and decides whether to search in the left half or the right half. This reduces the problem size by half in each recursive call. The solution does not require combining results from both halves; it simply continues with one half. Because the input size keeps reducing systematically, binary search is a clear example of Divide & Conquer.

Fibonacci (Memoized) – Dynamic Programming: The Fibonacci problem contains overlapping subproblems. In the naive version, the same Fibonacci values are calculated multiple times, which increases execution time. In the memoized version, previously computed values are stored in a dictionary and reused when needed. This avoids repeated calculations and improves efficiency from exponential to linear time. Since it solves subproblems once and stores their results, it follows the concept of Dynamic Programming.

Tower of Hanoi – Recursion: Tower of Hanoi is a classic example of recursion. The problem is solved by breaking it into smaller subproblems of the same type. To move n disks, the algorithm first moves n−1 disks, then moves the largest disk, and then again moves n−1 disks. The base case occurs when there is only one disk to move. This clearly shows how recursion works using smaller subproblems and a base condition.

Real-Life Examples of Paradigms:

- **Divide & Conquer:** Searching for a word in a dictionary by opening the middle page and deciding which half to search next.
- **Dynamic Programming:** Saving previously solved questions during exam preparation so they don't need to be solved again.
- **Greedy:** Giving change using the largest available currency denomination first without reconsidering previous choices.

Conclusion: Through this assignment, I understood how recursion works practically and how algorithm efficiency depends on implementation. The comparison between naive and memoized Fibonacci clearly showed the importance of optimization techniques like memoization. I also learned how divide and conquer reduces problem size efficiently, as seen in binary search, and how recursion works step by step in Tower of Hanoi. Overall, this assignment helped me connect theoretical concepts like Big-O notation with real coding implementation.